МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

ОТЧЕТ ПО ЛАБОРАТОРНЫМ РАБОТАМ
по дисциплине “Хранилища данных”

Учебная группа 223-321

Студент
Петренко Ю.И.

Москва
2026

ОГЛАВЛЕНИЕ

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине “Хранилища данных”.....3
Анализ предметной области и выделение сущностей.....4
Модель Data Vault.....4
Приложение 1. SQL проверка работоспособности модели Data Vault.....6
ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине “Хранилища данных”.....9
Работа с системой контроля версий Git.....10
ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине “Хранилища данных”.....11
4.1-4.2 Конфигурация MongoDB:.....12
4.2: Клиент MongoDB.....12
4.3.2 Запросы.....13
4.3.2.1 Одноуровневый поиск.....13
4.3.2.2 Многоуровневый поиск.....15
4.3.3 Удаление.....16
ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине “Хранилища данных”.....17
Первичный анализ.....18
Формирование групп пользователей.....18
Описание персонажей.....19
Структура графа.....20
Реализация графа на языке Cypher.....21
Код.....21
Результат в Neo4j.....22

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине “Хранилища данных”

«Проектирование хранилища данных (Data Vault)»
вариант №14 “Система учета в видеопрокате”

Учебная группа 223-321

	Должность	ФИО	Дата	Подпись
Выполнил	**Студент**	**Петренко Ю.И.**		
Проверил	**Преподаватель**	**Ноздрин В.С.**		

Москва
2026

Анализ предметной области и выделение сущностей

В видеопрокате **ключевыми бизнес-процессами** являются выдача фильмов и их возврат. Основные сущности:

- **Клиент —** человек, берущий фильм.
- **Фильм —** конкретная физическая копия (диск/кассета) или цифровой контент.
- **Сотрудник —** работник, оформляющий сделку.
- **Аренда —** факт транзакции (связь клиента, фильма и сотрудника).

Модель Data Vault

Хабы (Hub) — уникальные бизнес-ключи

- **hub_customer:** Бизнес-ключ — customer_email (уникален и неизменен). Метаданные: *customer_hash_key (PK), load_date, record_source.*
- **hub_film:** Бизнес-ключ — film_sku (штрих-код диска). Метаданные: film_hash_key (PK), load_
- **hub_employee:** Бизнес-ключ — employee_tab_num (табельный номер). Метаданные: *employee_hash_key (PK), load_date, record_source.*

Ссылки (Link) — связи многие-ко-многим

- **link_rental:** Связывает Клиента, Фильм и Сотрудника. Бизнес-ключ — rental_receipt_num (номер чека/договора).
  Метаданные: *rental_hash_key (PK), load_date, record_source, а также внешние ключи customer_hash_key, film_hash_key, employee_hash_key.*

Сателлиты (Satellite) — описательные атрибуты и история

- **sat_customer_details:** Привязан к hub_customer. Атрибуты: Имя, Фамилия, Телефон, Адрес, Дата рождения, Статус (VIP/обычный). Включает hash_diff для отслеживания изменений.
- **sat_film_details:** Привязан к hub_film. Атрибуты: Название, Год выпуска, Жанр, Режиссер, Формат (DVD/Blu-Ray), Длительность.
- **sat_rental_details:** Привязан к link_rental. Атрибуты: Дата выдачи, Плановая дата возврата, Фактическая дата возврата, Стоимость аренды, Сумма штрафа, Статус (выдан/возвращен/просрочен).

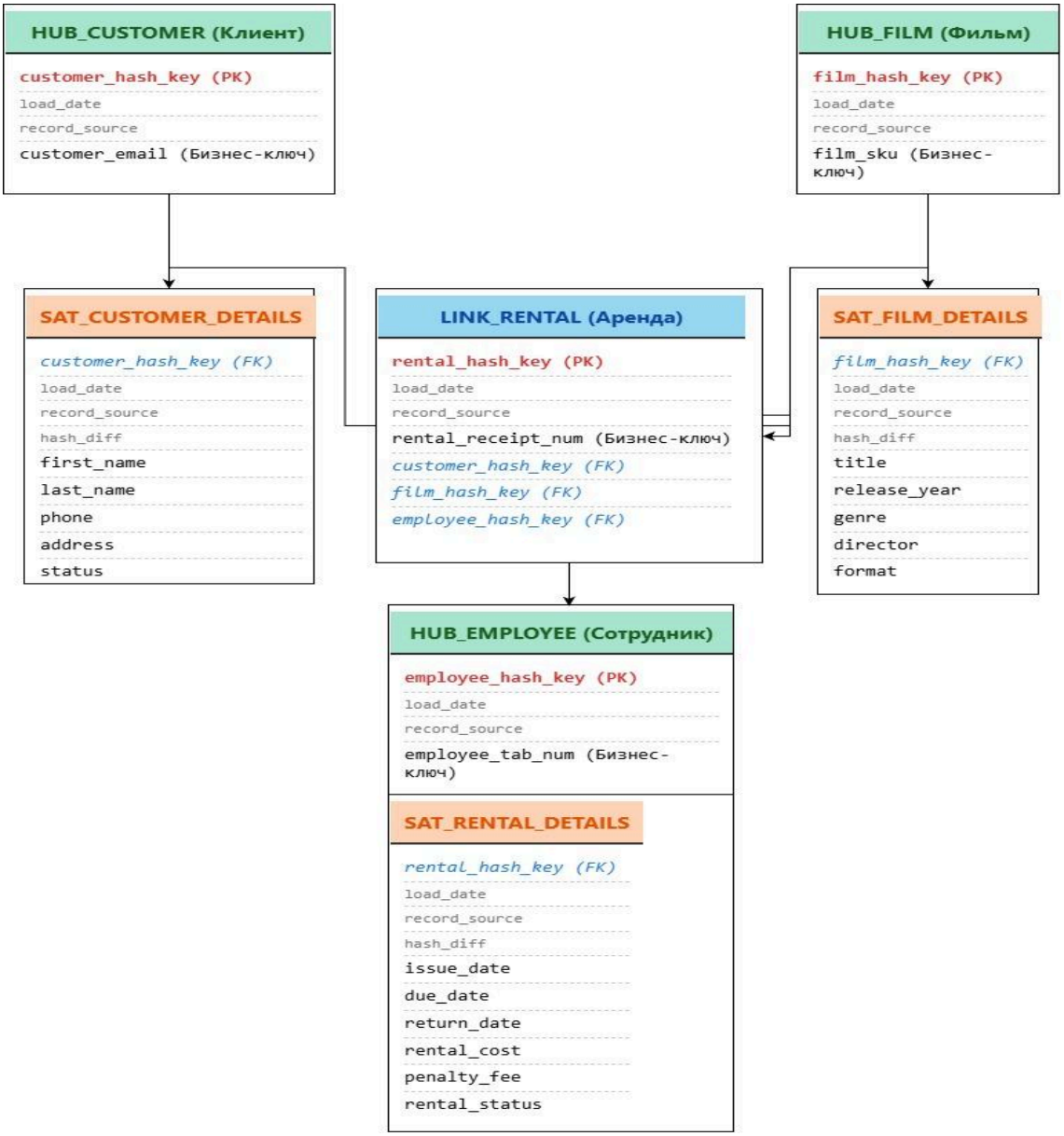


рисунок 1. Модель DataVault

Как читать схему:

■ Зеленые (Hubs) — содержат уникальные бизнес-ключи (email клиента, SKU фильма, табельный номер сотрудника).

■ Синие (Links) — связывают Хабы. *Link_Rental* связывает Клиента, Фильм и Сотрудника.

■ Оранжевые (Satellites) — содержат историю изменений (имена, телефоны, даты выдачи, штрафы).

Приложение 1. SQL проверка работоспособности модели Data Vault

Запрос:

```sql
-- ХАБЫ (Hub)
CREATE TABLE hub_customer (
    customer_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    customer_email VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE hub_film (
    film_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    film_sku VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE hub_employee (
    employee_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    employee_tab_num VARCHAR(20) NOT NULL UNIQUE
);

-- ССЫЛКИ (Link)
CREATE TABLE link_rental (
    rental_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    rental_receipt_num VARCHAR(50) NOT NULL UNIQUE,
    customer_hash_key CHAR(32) NOT NULL,
    film_hash_key CHAR(32) NOT NULL,
    employee_hash_key CHAR(32) NOT NULL,
    FOREIGN KEY (customer_hash_key) REFERENCES hub_customer(customer_hash_key),
    FOREIGN KEY (film_hash_key) REFERENCES hub_film(film_hash_key),
    FOREIGN KEY (employee_hash_key) REFERENCES hub_employee(employee_hash_key)
);

-- САТЕЛЛИТЫ (Satellite)
CREATE TABLE sat_customer_details (
    customer_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    hash_diff CHAR(32) NOT NULL,
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    phone VARCHAR(20),
    address TEXT,
    birth_date DATE,
```

```
    status VARCHAR(20)
);

CREATE TABLE sat_film_details (
    film_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    hash_diff CHAR(32) NOT NULL,
    title VARCHAR(255) NOT NULL,
    release_year INT,
    genre VARCHAR(100),
    director VARCHAR(255),
    format VARCHAR(50),
    duration_min INT
);

CREATE TABLE sat_rental_details (
    rental_hash_key CHAR(32) PRIMARY KEY,
    load_date TIMESTAMP NOT NULL,
    record_source VARCHAR(100) NOT NULL,
    hash_diff CHAR(32) NOT NULL,
    issue_date TIMESTAMP NOT NULL,
    due_date TIMESTAMP NOT NULL,
    return_date TIMESTAMP,
    rental_cost DECIMAL(10, 2),
    penalty_fee DECIMAL(10, 2) DEFAULT 0,
    rental_status VARCHAR(50)
);

-- 2. НАПОЛНЕНИЕ ДАННЫМИ (INSERT)
-- Хабы
INSERT INTO hub_customer VALUES ('hash_cust_1', CURRENT_TIMESTAMP, 'CRM_System',
'ivanov@mail.ru');
INSERT INTO hub_customer VALUES ('hash_cust_2', CURRENT_TIMESTAMP, 'CRM_System',
'petrova@yandex.ru');

INSERT INTO hub_film VALUES ('hash_film_1', CURRENT_TIMESTAMP, 'Inventory_System',
'SKU-001-DVD');
INSERT INTO hub_film VALUES ('hash_film_2', CURRENT_TIMESTAMP, 'Inventory_System',
'SKU-002-BD');

INSERT INTO hub_employee VALUES ('hash_emp_1', CURRENT_TIMESTAMP, 'HR_System',
'EMP-001');

-- Ссылки (Аренда)
INSERT INTO link_rental VALUES ('hash_rent_1', CURRENT_TIMESTAMP, 'POS_Terminal',
'REC-1001', 'hash_cust_1', 'hash_film_1', 'hash_emp_1');

-- Сателлиты
INSERT INTO sat_customer_details VALUES ('hash_cust_1', CURRENT_TIMESTAMP,
'CRM_System', 'diff_c1', 'Иван', 'Иванов', '+79001112233', 'ул. Пушкина, 1',
```

```
'1990-01-01', 'VIP');
INSERT INTO sat_customer_details VALUES ('hash_cust_2', CURRENT_TIMESTAMP,
'CRM_System', 'diff_c2', 'Анна', 'Петрова', '+79004445566', 'ул. Ленина, 2',
'1995-05-15', 'Обычный');

INSERT INTO sat_film_details VALUES ('hash_film_1', CURRENT_TIMESTAMP,
'Inventory_System', 'diff_f1', 'Матрица', 1999, 'Фантастика', 'Сестры Вачовски',
'DVD', 136);

INSERT INTO sat_rental_details VALUES ('hash_rent_1', CURRENT_TIMESTAMP,
'POS_Terminal', 'diff_r1', '2023-10-25 10:00:00', '2023-10-28 10:00:00', NULL,
150.00, 0.00, 'Выдан');

-- 3. ПРОВЕРКА РЕЗУЛЬТАТА (Выборка данных)
SELECT * FROM hub_customer;
SELECT * FROM link_rental;
SELECT * FROM sat_rental_details;
```

Ответ:

customer_hash_key	load_date	record_source	customer_email
hash_cust_1	2026-10-26 07:35:06	CRM_System	ivanov@mail.ru
hash_cust_2	2026-10-26 07:35:06	CRM_System	petrova@yandex.ru

рисунок 2. SQL ответ

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

ЛАБОРАТОРНАЯ РАБОТА №2
по дисциплине “Хранилища данных”

«Работа с системой контроля версий Git»

Учебная группа 223-321

	Должность	ФИО	Дата	Подпись
Выполнил	**Студент**	**Петренко Ю.И.**		
Проверил	**Преподаватель**	**Ноздрин В.С.**		

Москва
2026

Работа с системой контроля версий Git

Цель работы: Освоить базовые навыки работы с системой контроля версий Git и платформой GitHub, научиться создавать удаленные репозитории и загружать в них файлы проектов.

Ход работы:

- Был создан аккаунт на платформе GitHub (или использован существующий).
- Создан новый публичный репозиторий с названием lw_datavault_223-321.
- Локально на компьютере была инициализирована папка с материалами лабораторной работы №1 (отчет, схема Data Vault, SQL-скрипты) при помощи команды git init.
- Выполнена привязка локального репозитория к удаленному: git remote add origin https://github.com/Ezegx/lw_datavault_223-321.git.
- Файлы добавлены в индекс (git add .), зафиксированы (git commit -m "Лабораторная 1") и отправлены на сервер (git push -u origin main).

Результат работы:
Все материалы лабораторной работы №1 успешно загружены в удаленный репозиторий.
Ссылка на репозиторий: https://github.com/Ezegx/lw_datavault_223-321

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине “Хранилища данных”

«MongoDB»

Учебная группа 223-321

	Должность	ФИО	Дата	Подпись
Выполнил	**Студент**	**Петренко Ю.И.**		
Проверил	**Преподаватель**	**Ноздрин В.С.**		

Москва
2026

Цель работы: Освоить базовые навыки работы с MongoDB

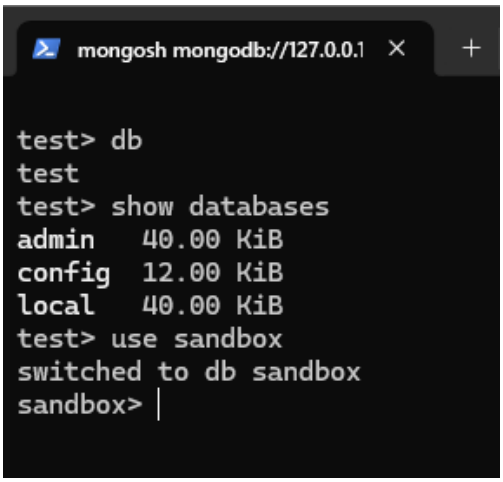
Ход работы:

4.1-4.2 Конфигурация MongoDB:

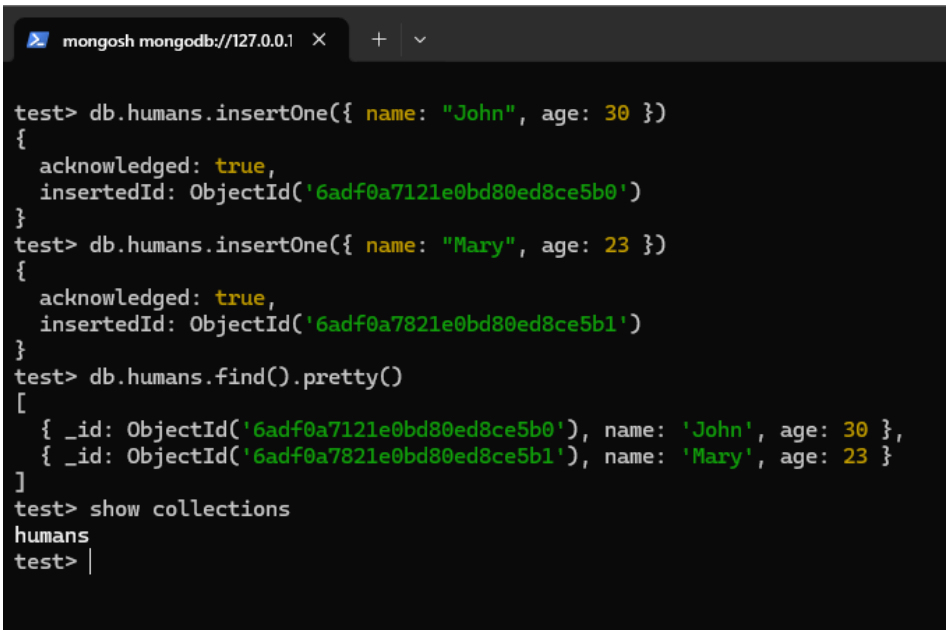
Для выполнения лабораторной работы будет использован контейнер **в Docker:**

1. **Скачивание официального образа MongoDB из DockerHUB:**
   ```
   docker pull mongodb/mongodb-community-server:latest
   ```
2. **Запуск контейнера:**
   ```
   docker run --name mongodb -p 27017:27017 -d mongodb/mongodb-community-server:latest
   ```
3. **Подключение к контейнеру:**
   ```
   docker exec -it mongodb mongosh
   ```

4.2: Клиент MongoDB



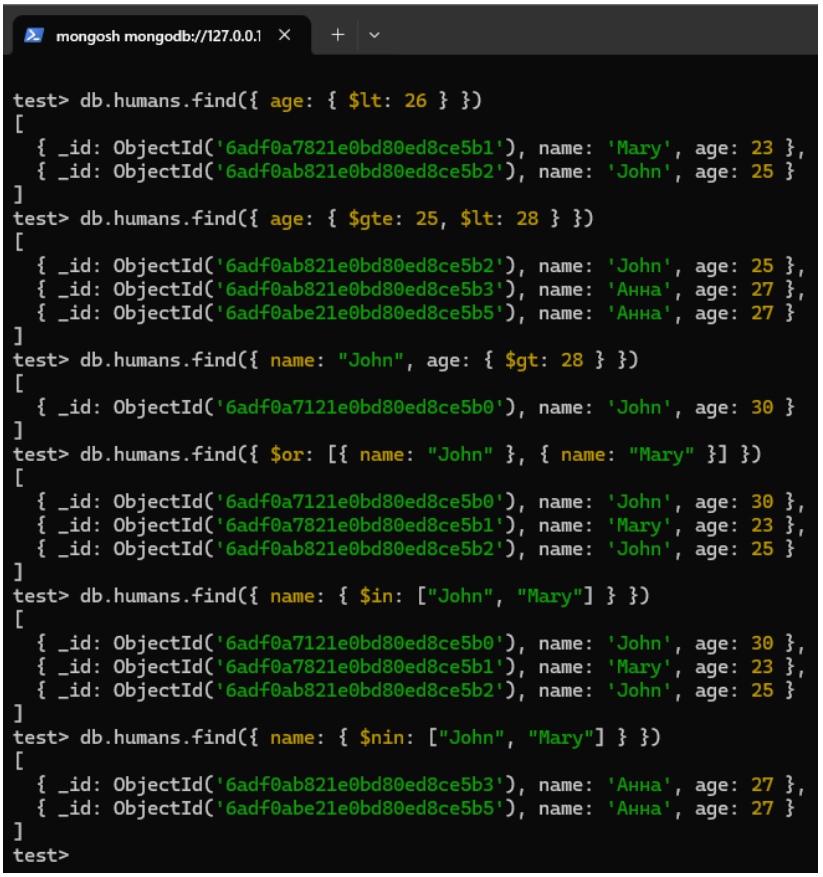
4.3.2 Запросы



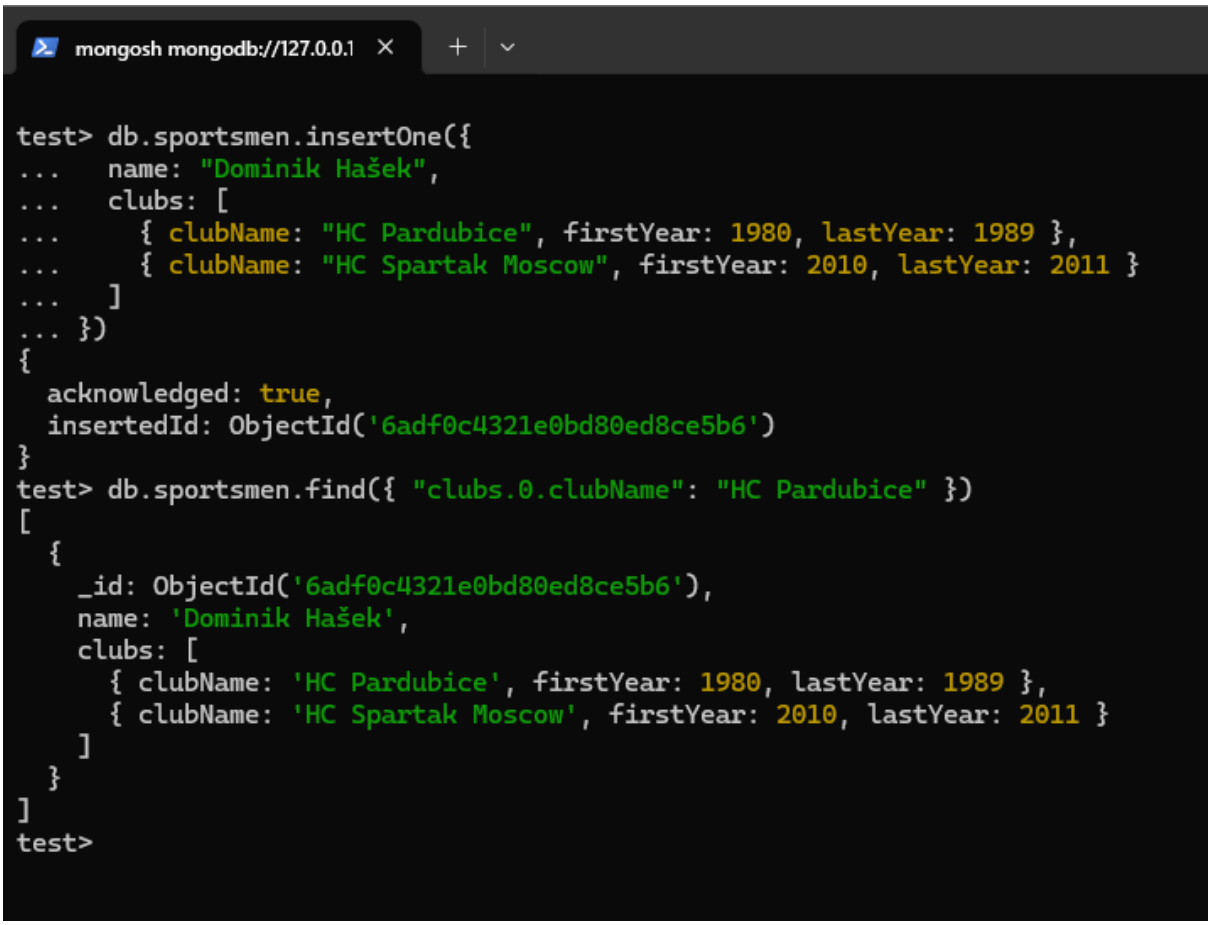
4.3.2.1 Одноуровневый поиск





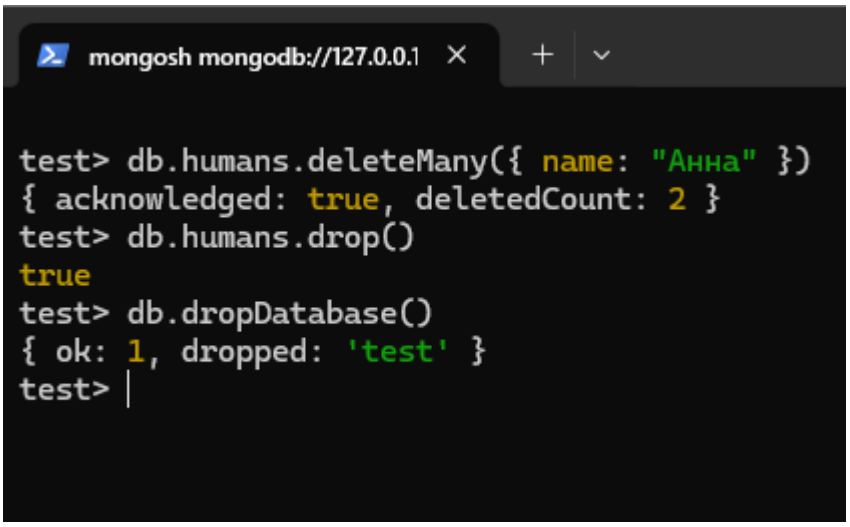

mongosh mongodb://127.0.0.1
test> db.humans.find({ age: { $lt: 26 } })
[
{ _id: ObjectId('6adf0a7821e0bd80ed8ce5b1'), name: 'Mary', age: 23 },
{ _id: ObjectId('6adf0ab821e0bd80ed8ce5b2'), name: 'John', age: 25 }
]
test> db.humans.find({ age: { $gte: 25, $lt: 28 } })
[
{ _id: ObjectId('6adf0ab821e0bd80ed8ce5b2'), name: 'John', age: 25 },
{ _id: ObjectId('6adf0ab821e0bd80ed8ce5b3'), name: 'Анна', age: 27 },
{ _id: ObjectId('6adf0abe21e0bd80ed8ce5b5'), name: 'Анна', age: 27 }
]
test> db.humans.find({ name: "John", age: { $gt: 28 } })
[
{ _id: ObjectId('6adf0a7121e0bd80ed8ce5b0'), name: 'John', age: 30 }
]
test> db.humans.find({ $or: [{ name: "John" }, { name: "Mary" }] })
[
{ _id: ObjectId('6adf0a7121e0bd80ed8ce5b0'), name: 'John', age: 30 },
{ _id: ObjectId('6adf0a7821e0bd80ed8ce5b1'), name: 'Mary', age: 23 },
{ _id: ObjectId('6adf0ab821e0bd80ed8ce5b2'), name: 'John', age: 25 }
]
test> db.humans.find({ name: { $in: ["John", "Mary"] } })
[
{ _id: ObjectId('6adf0a7121e0bd80ed8ce5b0'), name: 'John', age: 30 },
{ _id: ObjectId('6adf0a7821e0bd80ed8ce5b1'), name: 'Mary', age: 23 },
{ _id: ObjectId('6adf0ab821e0bd80ed8ce5b2'), name: 'John', age: 25 }
]
test> db.humans.find({ name: { $nin: ["John", "Mary"] } })
[
{ _id: ObjectId('6adf0ab821e0bd80ed8ce5b3'), name: 'Анна', age: 27 },
{ _id: ObjectId('6adf0abe21e0bd80ed8ce5b5'), name: 'Анна', age: 27 }
]
test>

4.3.2.2 Многоуровневый поиск



Ключ "clubs.0.clubName" берется в кавычки, потому что он содержит точки, и без кавычек JavaScript попытался бы интерпретировать это как вызов свойств объекта (clubs -> 0 -> clubName), что привело бы к синтаксической ошибке.

4.3.3 Удаление



Результат работы:
Все материалы лабораторной работы №3 успешно изучены и выполнены.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

ЛАБОРАТОРНАЯ РАБОТА №4
по дисциплине “Хранилища данных”

«Разработка структуры социального графа на основе алгоритма описания персонажей»
вариант №14 “Видеопрокат”

Учебная группа 223-321

	Должность	ФИО	Дата	Подпись
Выполнил	**Студент**	**Петренко Ю.И.**		
Проверил	**Преподаватель**	**Ноздрин В.С.**		

Москва
2026

Цель работы: Изучить методологию создания пользовательских персонажей и применить её для проектирования графовой базы данных. Разработать структуру социального графа (не менее 10 узлов и 30 рёбер) и реализовать её на языке запросов Cypher.

Первичный анализ

В качестве предметной области выбрана система учета видеопроката (как физического, так и цифрового).

- **Кто пользователи:** Клиенты проката (разного возраста и социального статуса), а также сотрудники пункта выдачи.
- **Что они делают:** Берут фильмы напрокат, возвращают их, оставляют оценки, вступают в клубы любителей кино, общаются с другими клиентами.
- **Результат анализа:** Пользователи делятся на группы в зависимости от их целей и активности.

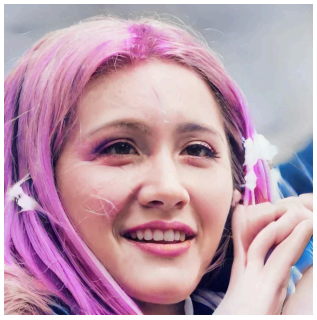
Формирование групп пользователей

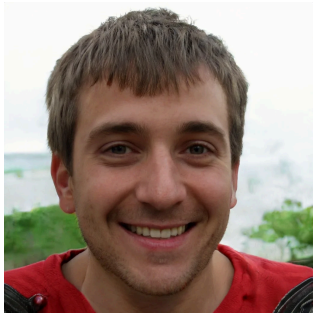
Разделим аудиторию по уникальному признаку — **цели посещения видеопроката:**

- **«Конечные потребители» (Обычные зрители).** Берут фильмы на выходные, чтобы просто развлечься. Редко пишут отзывы.
- **«Киноманы (Создатели контента)».** Активно обсуждают фильмы, пишут рецензии, дают рекомендации другим.
- **«Профессионалы (Сотрудники и Админы клубов)».** Управляют выдачей дисков, модерируют тематические клубы.

Описание персонажей

На основе выделенных групп составим 5 конкретных персонажей:

	**Юлия (17 лет, Москва). Группа: Конечные потребители.** **Биография: Учится на первом курсе колледжа на дизайнера. Живёт с родителями, много времени проводит в социальных сетях. В кино ходит редко, чаще смотрит фильмы дома по выходным. Не любит сложные сюжеты, предпочитает лёгкие комедии.** **Цель: Найти интересные фильмы для вечернего просмотра с друзьями, брать диски в прокате на выходные.**
	**Марк (28 лет, Санкт-Петербург). Группа: Киноманы.** **Биография: Работает инженером-программистом. Окончил политехнический университет. В свободное время увлекается режиссурой, пишет обзоры на фильмы в своём блоге. Регулярно посещает артхаусные показы.** **Цель: Находить редкие фильмы в прокате, оставлять развернутые рецензии и рекомендовать их друзьям.**
	**Анна (22 года, Казань). Группа: Киноманы.** **Биография: Студентка-социолог, активистка. Состоит в клубе любителей фантастики. Обожает обсуждать сюжетные повороты, спорить о концовках и делиться впечатлениями в тематических сообществах.** **Цель: Смотреть новинки проката, обсуждать их в клубе и рекомендовать друзьям.**

	**Олег (19 лет, Москва). Группа: Конечные потребители.** **Биография: Студент технического вуза. Живёт в общежитии. В свободное время играет в видеоигры или смотрит фильмы ужасов с соседями. В прокат ходит нерегулярно, но всегда берёт что-то «пострашнее».** **Цель: Провести вечер с друзьями за просмотром фильма, поделиться впечатлениями.**
	**Елена (25 лет, Санкт-Петербург). Группа: Профессионалы.** **Биография: Имеет высшее образование в сфере менеджмента. Работает в видеопрокате уже 3 года, начинала с кассира, сейчас управляет точкой. Отвечает за выдачу дисков и порядок в зале.** **Цель: Эффективно управлять прокатом, следить за возвратами, помогать клиентам с выбором.**

Структура графа

В отличие от обычной реляционной базы данных, граф покажет связи между людьми и объектами видеопроката.

Узлы:

- Client (Клиент): свойства name, age, city.
- Film (Фильм): свойства title, genre, year.
- Employee (Сотрудник): свойство name, position.

Связи:

- FRIEND (Дружба): между двумя Client.
- RENTED (Взял в прокат): от Client к Film (свойство date — дата выдачи).
- RECOMMENDED (Рекомендовал): от Client к Film (свойство rating — оценка).
- ISSUED (Выдал): от Employee к Film (в рамках рабочего процесса).
- LIKES_FILM (Понравился фильм): от Client к Film.

Реализация графа на языке Cypher

Код

```
CREATE
  (julia:Client {name: 'Юлия', age: 17, city: 'Москва'}),
  (mark:Client {name: 'Марк', age: 28, city: 'СПб'}),
  (anna:Client {name: 'Анна', age: 22, city: 'Казань'}),
  (oleg:Client {name: 'Олег', age: 19, city: 'Москва'}),
  (elena:Employee {name: 'Елена', position: 'Старший менеджер'}),
  (f_comedy:Film {title: 'Мальчишник в Вегасе', genre: 'Комедия', year:
2009}),
  (f_arthouse:Film {title: 'Матрица', genre: 'Фантастика', year: 1999}),
  (f_horror:Film {title: 'Звонок', genre: 'Ужасы', year: 2002}),

  (emp_mark:Employee {name: 'Марк', position: 'Кассир'}),
  (g_club:Club {name: 'Клуб любителей фантастики', topic: 'Sci-Fi'})
CREATE (julia)-[:FRIEND]->(mark)
CREATE (julia)-[:FRIEND]->(anna)
CREATE (mark)-[:FRIEND]->(oleg)
CREATE (anna)-[:FRIEND]->(oleg)
CREATE (mark)-[:FRIEND]->(anna)
CREATE (oleg)-[:FRIEND]->(julia)
CREATE (julia)-[:RENTED {date: '2023-10-25'}]->(f_comedy)
CREATE (mark)-[:RENTED {date: '2023-10-26'}]->(f_arthouse)
CREATE (anna)-[:RENTED {date: '2023-10-26'}]->(f_arthouse)
CREATE (oleg)-[:RENTED {date: '2023-10-27'}]->(f_horror)
CREATE (julia)-[:RENTED {date: '2023-10-28'}]->(f_horror)
CREATE (mark)-[:RENTED {date: '2023-10-29'}]->(f_arthouse)
CREATE (anna)-[:RENTED {date: '2023-10-30'}]->(f_comedy)
CREATE (mark)-[:RECOMMENDED {rating: 10}]->(f_arthouse)
CREATE (anna)-[:RECOMMENDED {rating: 9}]->(f_arthouse)
CREATE (mark)-[:RECOMMENDED {rating: 8}]->(f_comedy)
CREATE (oleg)-[:RECOMMENDED {rating: 7}]->(f_horror)
CREATE (julia)-[:RECOMMENDED {rating: 6}]->(f_comedy)
CREATE (anna)-[:RECOMMENDED {rating: 10}]->(f_horror)
CREATE (julia)-[:LIKES_FILM]->(f_comedy)
CREATE (oleg)-[:LIKES_FILM]->(f_comedy)
CREATE (mark)-[:LIKES_FILM]->(f_arthouse)
CREATE (anna)-[:LIKES_FILM]->(f_horror)
CREATE (oleg)-[:LIKES_FILM]->(f_horror)
CREATE (elena)-[:ISSUED]->(f_comedy)
CREATE (elena)-[:ISSUED]->(f_arthouse)
CREATE (emp_mark)-[:ISSUED]->(f_horror)
CREATE (emp_mark)-[:ISSUED]->(f_comedy)
CREATE (mark)-[:MEMBER_OF]->(g_club)
CREATE (anna)-[:MEMBER_OF]->(g_club);
```

Результат в Neo4j

